Brane Enterprises — AI Agent Runtime for a No-Code Platform — Complete Deep Dive

PostgreSQL · gRPC model gateway · Attribute-level RBAC · Verifier loop · Tool invocation · MCP routing

Architecture · Implementation · Design Decisions · Interview Prep · Resume Bullet

---

PART 1: WHAT THIS IS AND WHY IT WAS BUILT

The context

Brane runs a **no-code platform** where business users compose workflows out of **Local Objectives (LOs)** — units of work. Until this project, every LO was one of two types:

- **HUMAN** — a person does the work (fills a form, approves something).
- **DIGITAL** — deterministic automation (fetch records, run a calculation, call an API).

Both are **deterministic**: given the same inputs they always do the same thing, and their output is trusted the moment it's produced.

The company wanted a third type: **AI_AGENT** — an LO where an LLM does the work (extract invoice fields, generate code, classify, summarize, plan). That sounds like "just add an LLM call," but it isn't, and the reason is the whole point of the project.

The problem it solved

An AI agent has a probabilistic core. It can be wrong, it can hallucinate, it can return malformed output, and it can try to touch data it shouldn't. You cannot drop a probabilistic component into a workflow engine that assumes every step is trustworthy the instant it runs — you'd be committing unreviewed, possibly-wrong AI output straight into business records.

Before this runtime existed, there was no safe way to:

- Let an AI agent produce a value **and prove it's acceptable before it's committed**.
- Stop an AI agent from **reading or writing data outside its scope** (an invoice extractor shouldn't see salary fields).
- Give the agent **tools** (fetch data, call an NF) without letting it invent uncontrolled actions.
- **Retry intelligently** when the output is wrong, feeding the failure reason back in — instead of failing the whole workflow.
- Keep a full **audit trail** of every attempt for a regulated enterprise.

What the runtime does

It introduces the AI_AGENT LO type with a **"probabilistic core, deterministic gate"** design:

- The AI agent generates output (the probabilistic part).
- A **separate verifier lane** — deterministic business rules → optional AI verifier → optional human verifier — decides whether that output is acceptable (the deterministic gate).
- On **pass**, the workflow's real primary function (create/update/etc.) fires and the LO completes.
- On **fail**, the failure report is fed back as extra context and the agent tries again — an **in-place correction loop** with a hard iteration cap.
- Throughout, **attribute-level RBAC** filters exactly which data the agent can see and write, and a **tool registry** limits which pre-registered functions it can invoke.
- Everything is **audited** — every attempt, every tool call, every verdict — in dedicated sibling tables.

What makes this different from "just call an LLM in a workflow"

1. The verifier is a separate execution lane, never the same call as generation. The thing that generates is never the thing that certifies. This separation is what makes the agent's output trustworthy enough to commit into business data.

2. Attribute-level RBAC, not just object-level. Existing platform RBAC controlled access at the LO level ("can this principal run this LO?"). AI agents need finer control — "this agent may read attribute `E1.At5` but not write it, and may not even see the salary attribute." That's permission down to the individual field.

3. In-place correction loop with attempt-sequence audit. Humans on the platform recurse by creating new versioned LO ids (LO4.1, LO4.2). An AI agent iterating 5 times in 3 seconds can't spawn 5 versioned LOs — so attempts are recorded in-place with an `attempt_seq`, every one auditable, the latest mirrored to the canonical table downstream readers use.

4. Nothing existing was altered. HUMAN and DIGITAL execution paths, and the canonical execution tables, were left untouched. All new state lives in **sibling tables** (suffixed `_agent`) so the AI runtime is additive, reversible, and immediately distinguishable.

Results / impact

- Delivered a **production AI agent runtime** that plugs into the existing engine through a single delegation point — an LO with a primary AI agent routes to the new controller; everything else flows through unchanged code.
- **Attribute-level RBAC caching, tool-invocation overhead reduction, and response-parser hardening** for malformed LLM output — the performance and reliability work that made the runtime usable in production.
- A **gRPC model gateway** giving multi-provider routing (instruct / thinking / vision) with stateless horizontal scaling.

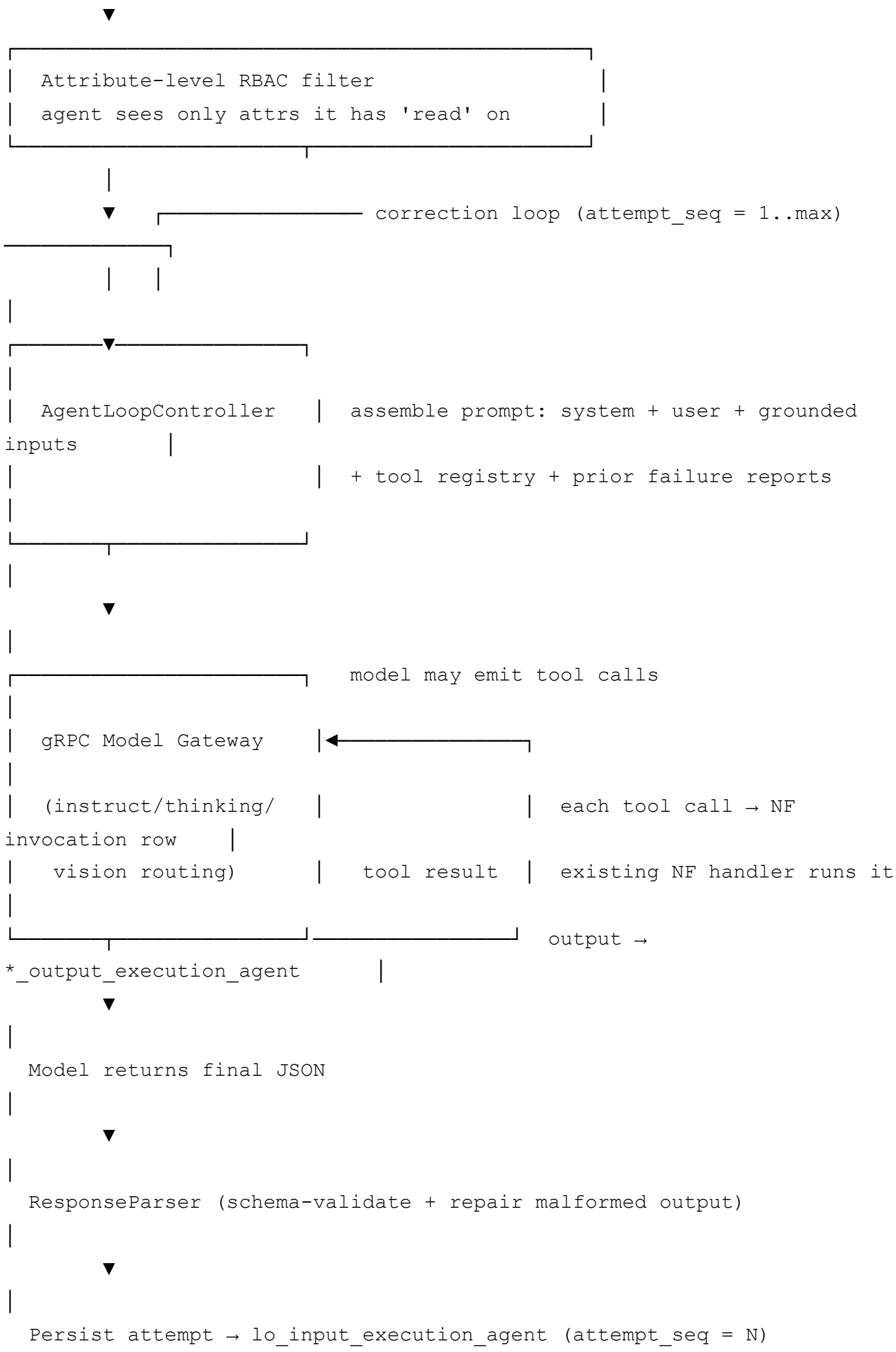
My role (be precise in interviews)

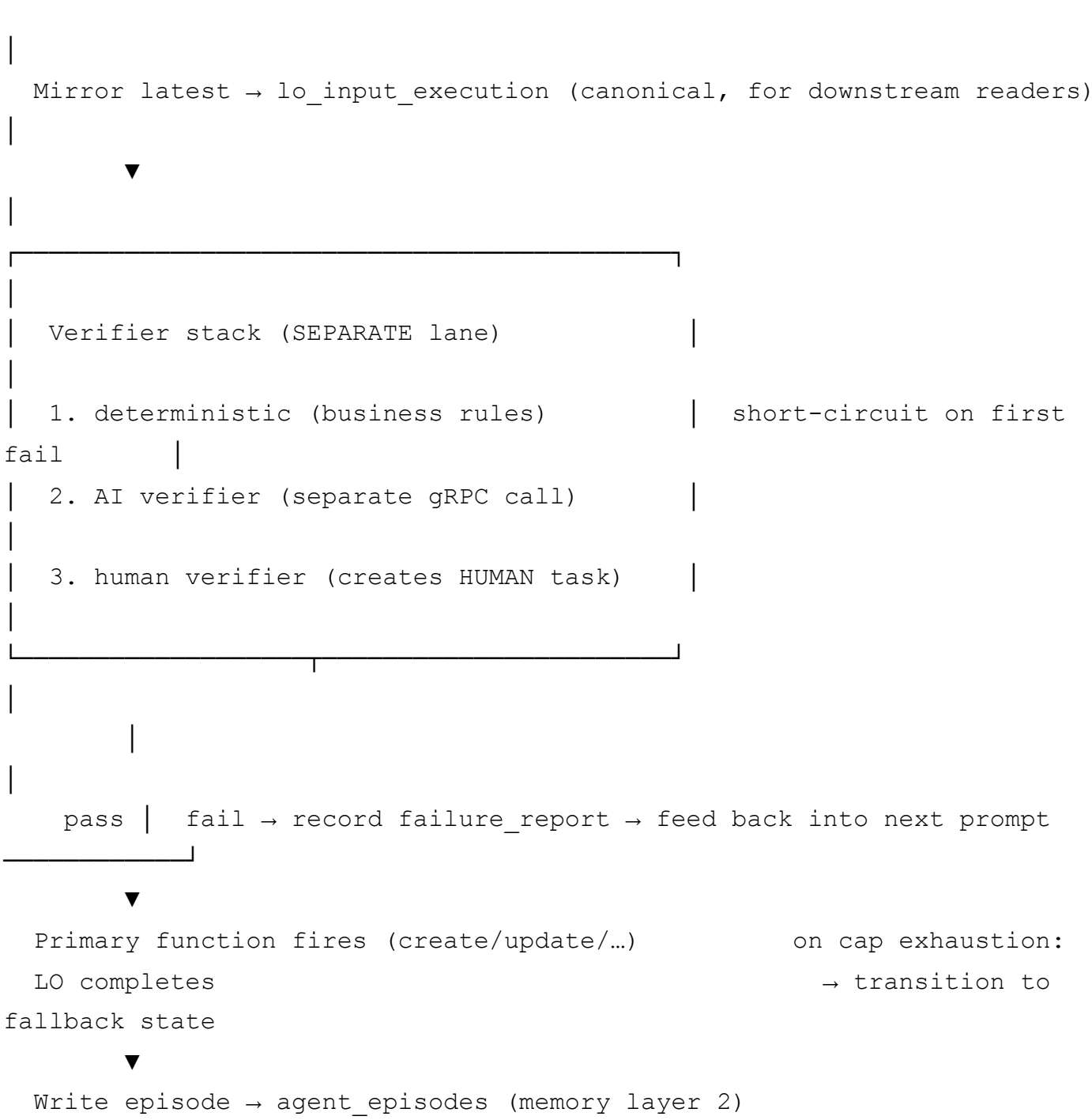
AI Engineer · Associate Solution Leader — 2 months. I contributed to the production runtime across: **PostgreSQL schema design** (agent identity, multi-agent junction, attribute-RBAC rows, execution sibling tables), **LLM inferencing via the gRPC model gateway**, **agent-loop development**, **prompt assembly**, and **performance/reliability optimization** (RBAC caching, tool overhead, parser hardening). It was a team effort on a large existing platform — I own my slices, and I understand the whole design.

PART 2: FULL ARCHITECTURE

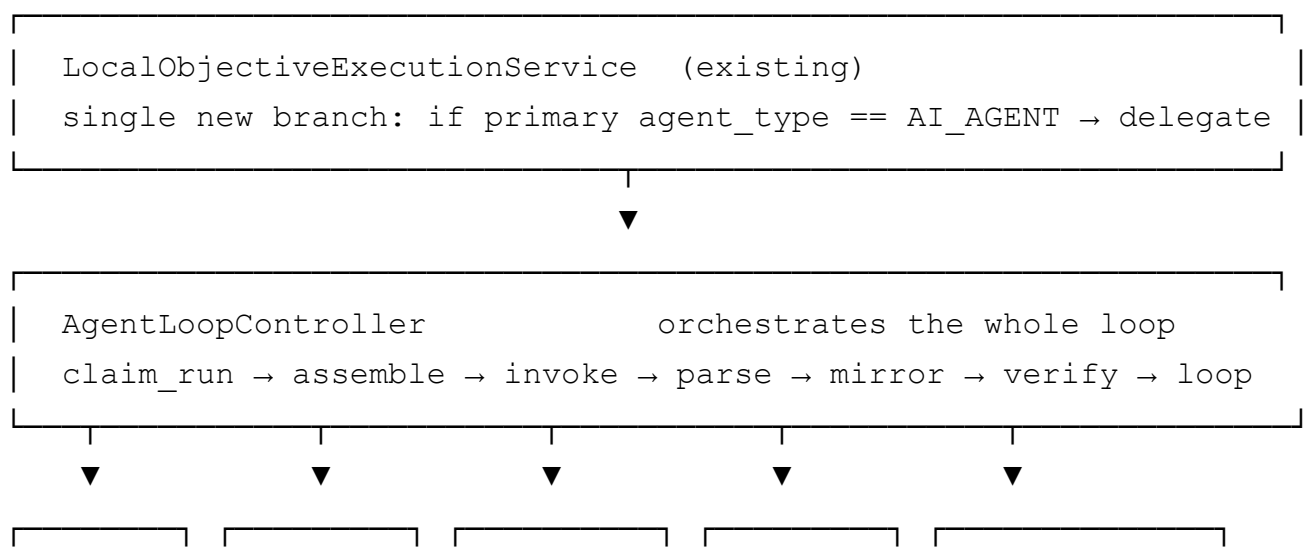
End-to-end execution flow

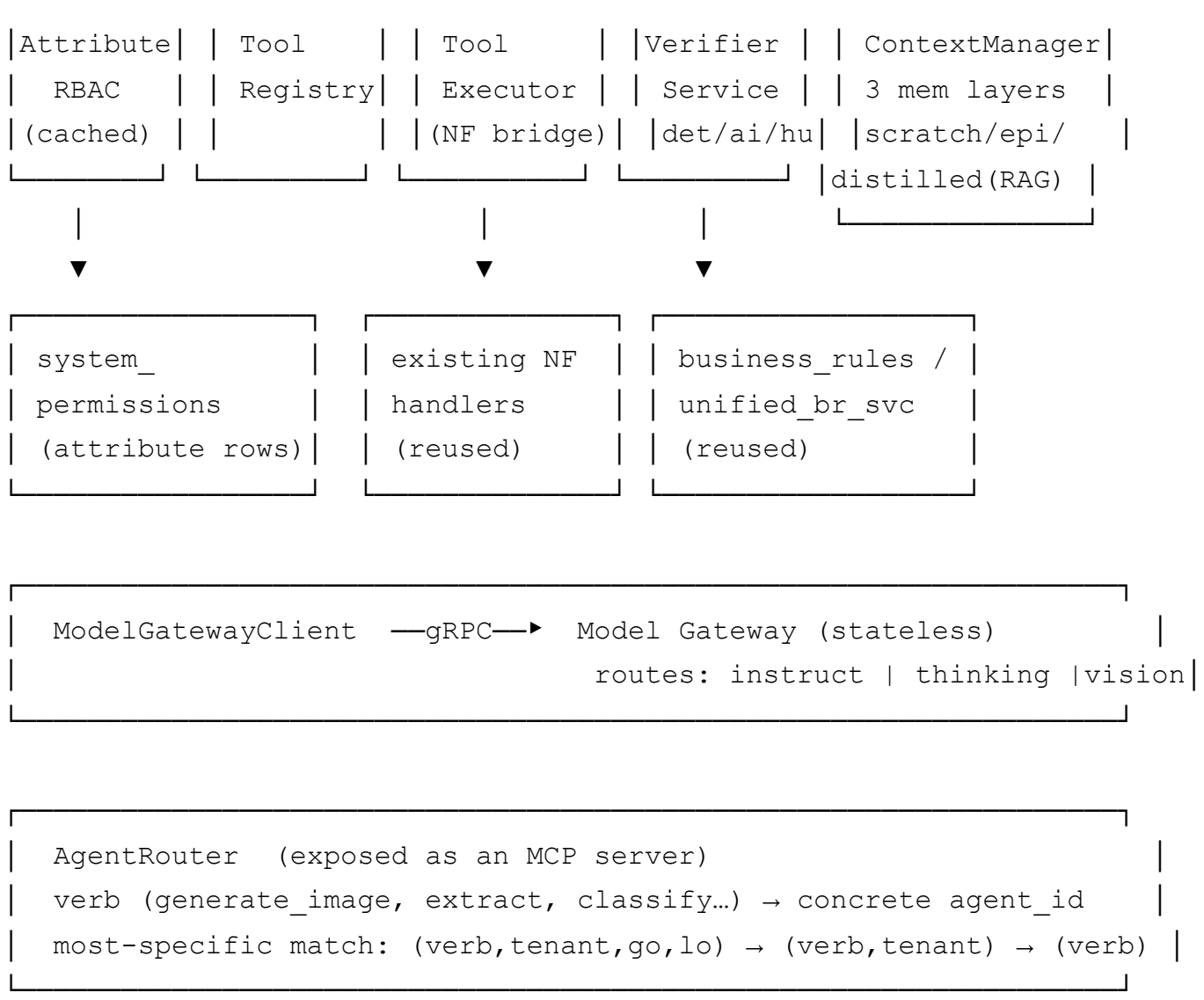
```
LO triggered (agent_type = AI_AGENT)
        |
        ▼
LO-level RBAC check (existing)                    ← unchanged platform gate
        |
        ▼
Design-time grounding NFs run (fetch, calc, ai_with_rag)
outputs → lo_nested_function_output_execution   ← existing tables,
untouched
        |
```


Attribute-level RBAC filter
agent sees only attrs it has 'read' on
correction loop (attempt_seq = 1..max)
AgentLoopController
assemble prompt: system + user + grounded inputs
+ tool registry + prior failure reports
model may emit tool calls
gRPC Model Gateway
(instruct/thinking/ vision routing)
tool result
each tool call → NF invocation row
existing NF handler runs it
output → *_output_execution_agent
Model returns final JSON
ResponseParser (schema-validate + repair malformed output)
Persist attempt → lo_input_execution_agent (attempt_seq = N)



Component diagram





The three memory layers

LAYER 1	SCRATCH (debug)	agent_run_scratch	reasoning trace + raw output, current run pruned after 7 days
LAYER 2	EPISODIC (learning)	agent_episodes	per (user × GO × LO) completed attempts inputs, outputs, verdict, attempt count; 90 days
LAYER 3	DISTILLED (knowledge)	RAG corpus + agent_rag_promotions	patterns promoted from episodic, admin-curated, requires N supporting episodes

---

PART 3: IMPLEMENTATION — KEY COMPONENTS

Schema — the sibling-table strategy

All new tables live in the `workflow_runtime` schema, suffixed `_agent` or prefixed `agent_` so they're immediately distinguishable. No existing table is structurally altered except clearly-marked additive columns.

```
-- Agent identity
CREATE TABLE workflow_runtime.agents (
    agent_id          VARCHAR(64) PRIMARY KEY,
    agent_name        VARCHAR(255) NOT NULL,
    agent_type        VARCHAR(32) NOT NULL,      -- HUMAN | DIGITAL | AI_
    tenant_id         VARCHAR(64) NOT NULL,
    model_class       VARCHAR(64),               -- 'instruct' | 'thinkir
    prompt_template_id VARCHAR(64),
    config            JSONB DEFAULT '{}'::jsonb,
    is_active         BOOLEAN DEFAULT TRUE,
    created_at        TIMESTAMP DEFAULT NOW()
);

-- LO ↔ agents junction (multi-agent: one primary, optional secondary/ver
CREATE TABLE workflow_runtime.lo_agents (
    id        SERIAL PRIMARY KEY,
    lo_id     VARCHAR(64) NOT NULL,
    agent_id  VARCHAR(64) NOT NULL REFERENCES workflow_runtime.agents(age
    role      VARCHAR(32) NOT NULL,              -- 'primary' | 'secondar
    priority  INT DEFAULT 100,
    is_active BOOLEAN DEFAULT TRUE,
    UNIQUE (lo_id, agent_id, role)
);
-- Exactly one active primary per LO — enforced by a partial unique index
CREATE UNIQUE INDEX idx_lo_agents_one_primary
    ON workflow_runtime.lo_agents(lo_id)
    WHERE role = 'primary' AND is_active = TRUE;

-- Every AI attempt (existing lo_input_execution keeps the current mirror
CREATE TABLE workflow_runtime.lo_input_execution_agent (
```

```
    id                  BIGSERIAL PRIMARY KEY,
    instance_id         UUID NOT NULL,
    lo_id               VARCHAR(64) NOT NULL,
    lo_input_item_id    VARCHAR(64) NOT NULL,
    attribute_id        VARCHAR(64),
    attempt_seq         INT NOT NULL,              -- 1-indexed per (instar
    written_by_agent_id VARCHAR(64) NOT NULL,
    input_value         JSONB,
    raw_model_output    TEXT,                      -- the raw response that
    verdict             VARCHAR(16),               -- pending|passed|failec
    failure_report      JSONB,                     -- populated if this att
    created_at          TIMESTAMP DEFAULT NOW(),
    UNIQUE (instance_id, lo_id, lo_input_item_id, attempt_seq)
);
```

Additive column on an existing table — the only kind of change made to canonical tables:

```
-- NULL = legacy behavior. When set, ordered list of permitted sources.
ALTER TABLE workflow_runtime.lo_input_items
    ADD COLUMN permitted_sources JSONB DEFAULT NULL;
-- ["ai_agent","human"] → AI fills first, human can override
-- ["user","fetch"]     → user fills first, fetch is fallback
```

Attribute-level permissions reuse the existing vertical `system_permissions` table — two additions: an `'attribute'` resource_type and a `'source'` permission_type alongside read/write/execute:

```
ALTER TYPE workflow_runtime.permission_type_enum ADD VALUE 'source';
CREATE INDEX idx_perms_attribute_lookup
    ON workflow_runtime.system_permissions(resource_type, resource_id, pr
    WHERE resource_type IN ('attribute', 'lo');
```

The loop — AgentLoopController (the heart)

The existing `LocalObjectiveExecutionService` delegates here when the primary agent is AI_AGENT. This is the retry-with-feedback correction loop.

```
class AgentLoopController:
    """
    1. claim_run()       — write ai_agent_run_state, attempt 1
    2. assemble_inputs() — RBAC-filtered input set
```

```
3. invoke_model()    — gRPC to gateway, handles tool calls
4. persist_attempt() — append to lo_input_execution_agent + mirror
5. run_verifier()    — deterministic → ai → human chain
6. pass → commit primary function ;  fail → feed failure back, loop
7. cap → transition_to_fallback()
8. write_episode()   — persist to agent_episodes
"""
def execute(self, instance_id, lo_id, user_id):
    agent_id = self._resolve_primary_agent(lo_id)
    state    = self._claim_run(instance_id, lo_id, agent_id)

    for attempt in range(1, state['max_iterations'] + 1):
        inputs  = self._assemble_inputs(instance_id, lo_id, agent_id,
        prompts = PromptBuilder(self.db).build(
            agent_id=agent_id, lo_id=lo_id, instance_id=instance_id,
            inputs=inputs,
            prior_failure_reports=self._prior_failures(instance_id, l
            attempt=attempt)
        tool_specs = self.tools.tools_for(agent_id, lo_id, instance_i

        result = self._invoke_with_tools(prompts, tool_specs, instanc
                                         lo_id, agent_id, attempt)
        parsed = ResponseParser().parse(result.final_text,
                                        self._output_schema_for(lo_ic
        if not parsed.ok:                       # malformed output = a f
            self._persist_attempt(..., verdict='failed',
                                  failure_report={'error': 'parse_fai
            if attempt >= state['max_iterations']: break
            continue

        self._persist_attempt(..., values=parsed.values, verdict='per
        self._mirror_to_canonical(instance_id, lo_id, parsed.values,

        verdict = self.verifier.verify(instance_id, lo_id, attempt, p
        if verdict.passed:
            self._write_episode(state, ..., 'committed')
            return {'status': 'converged', 'attempts_used': attempt,
                    'final_values': parsed.values}

        self._record_failure(instance_id, lo_id, attempt, verdict.fai

    # exhausted
    self._write_episode(state, ..., 'fallback')
    return self._transition_to_fallback(instance_id, lo_id, state)
```

The single delegation point that wires it into the existing engine — the only change to the platform's main path:

```
def execute_local_objective(self, user_id, tenant_id, instance_id, lo_id,
    if self._get_primary_agent_type(lo_id) == 'AI_AGENT':
        from app.api.v2.ai_agent.service import AgentLoopController
        return AgentLoopController(self.db, tenant_id).execute(instance_i
    # Existing HUMAN / DIGITAL path — unchanged.
    ...
```

Attribute-level RBAC (with the caching I optimized)

The agent only sees attributes it has `read` on; the per-instance cache is the optimization that stopped this becoming a per-field DB round-trip.

```
class AttributeRBAC:
    def __init__(self, db):
        self.db = db
        self._cache = {}                                # per-instance cache —

    def can_read(self, principal, attribute_id, tenant_id):
        return self._check(principal, attribute_id, 'read', tenant_id)

    def _check(self, principal, attribute_id, permission, tenant_id):
        ptype, pid = principal
        key = ('check', ptype, pid, attribute_id, permission, tenant_id)
        if key in self._cache:                          # avoid re-querying for
            return self._cache[key]
        row = self.db.execute(text("""
            SELECT 1 FROM workflow_runtime.system_permissions
            WHERE resource_type='attribute' AND resource_id=:rid
              AND permission_type=:perm AND tenant_id=:t
              AND ( (principal_type=:ptype AND principal_id=:pid)
                 OR (principal_type='role' AND principal_id IN (
                       SELECT role_id FROM workflow_runtime.principal_rol
                       WHERE principal_id=:pid AND principal_type=:ptype)
            LIMIT 1
        """), {...}).fetchone()
        result = row is not None
        self._cache[key] = result
        return result
```

Tool executor — bridging AI tool calls to existing NF handlers

The agent doesn't invent actions; it calls **pre-registered NFs** exposed as tools. Each call is grant-checked, schema-validated, audited in, dispatched to the *existing* handler, and audited out.

```
class ToolExecutor:
    def execute(self, call, instance_id, lo_id, agent_id, parent_attempt_
        tool = self._resolve_tool(call.tool_id)                    # → back
        if not tool:                          return {'error': 'unknown_tool
        if not self._is_granted(agent_id, call.tool_id):
                                              return {'error': 'tool_not_gra
        try:                                                       # jonsc
            validate(call.params, tool.parameter_schema)
        except ValidationError as e:          return {'error': 'param_valida

        seq = self._next_seq(instance_id, tool.backing_nf_id)      # 1-inde
        self._persist_input_rows(...)                              # audit

        try:                                                       # reuse
            output = self._dispatch(tool.function_type, tool.function_nam
                                    tool.backing_nf_id, call.params, inst
            status, error = 'success', None
        except Exception as e:
            output, status, error = None, 'error', str(e)

        self._persist_output_rows(..., status, error)              # audit
        return {'tool_id': call.tool_id, 'result': output, 'status': stat
```

Verifier service — the deterministic gate (separate lane)

Order: deterministic (reuses existing business rules) → AI verifier (separate gRPC call, different model class) → human verifier (creates a HUMAN LO task). **First failure short-circuits.**

```
class VerifierService:
    def verify(self, instance_id, lo_id, attempt_seq, values) -> Verifier
        for v in self._load_verifier_chain(lo_id):      # ordered det → ai
            verdict = self._run_one(v, instance_id, lo_id, attempt_seq, v
            self._persist_verdict(instance_id, lo_id, v, attempt_seq, ver
            if not verdict.passed:
```

```
                return verdict                                  # short-circuit c
        return VerifierVerdict(passed=True, verifier_type='none')

    def _run_deterministic(self, v, values, lo_id):
        from app.api.v2.business_rules.unified_br_service import Business
        result = BusinessRuleService(self.db).evaluate_for_lo(lo_id=lo_ic
        return VerifierVerdict(passed=result['valid'], verifier_type='det
                               failure_report=None if result['valid']
                               else {'rule_failures': result['rules_evalu
```

gRPC model gateway (multi-provider, stateless)

The runtime never calls a provider SDK directly. It calls one gRPC gateway that routes by `model_class` (instruct / thinking / vision). Because the gateway holds no per-request state, it scales by adding replicas behind a load balancer.

```
class ModelGatewayClient:
    """gRPC client wrapping the model gateway service.
       invoke(): main generation call, streams tool calls back to on_tool
       verify(): separate call used by the AI verifier — different model
    def invoke(self, system_prompt, user_prompt, grounded_inputs, tools,
               on_tool_call, agent_id, instance_id, lo_id): ...
```

Infra (from the dependency guide): a new `ai-gateway` service, workflow-engine → gateway on `50051/tcp` (gRPC), deps `grpcio`, `grpcio-tools`, `protobuf`, `jsonschema` (tool-param validation), and observability metrics `ai_agent_run_total`, `ai_agent_attempts_per_run`, `ai_agent_verifier_failures_total`, `ai_gateway_latency_ms`.

Verb-based agent routing via MCP

Designers shouldn't pin a model on an LO. They declare **what the LO is trying to do** — a `primary_verb` (`generate_image`, `extract`, `classify`, `summarize`, `plan` …) — and an **MCP-backed router** resolves `(verb, tenant, context) → concrete agent_id` at runtime. Result: the same LO routes to `imagegen_v3` today and `imagegen_v4` next quarter with **zero LO change**.

```
Resolution — most-specific match wins:
  1. Exact        (verb, tenant_id, go_id, lo_id)
  2. Tenant+GO    (verb, tenant_id, go_id)
```

```
  3. Tenant only (verb, tenant_id)
  4. Global      (verb)
  5. Default     agent_verbs.default_model_class  (or fail:
routing_unresolved)
Ties: higher priority, then most-recently-created. Context predicates:
{"region":"EU"} routes EU LOs to a region-pinned agent.
```

The router is exposed as an **MCP server** (tools: `resolve_agent`, ...) so design-time tools and the workflow engine speak the same protocol.

PART 4: DESIGN DECISIONS — WHY EACH CHOICE WAS MADE

Why a separate verifier lane, never the same call as generation?

This is the central design principle and the most likely deep question. **The component that generates must not be the component that certifies its own output.** If you ask the same model call "produce this AND tell me if it's right," a confidently-wrong answer certifies itself. By running verification as a **separate execution lane** — deterministic business rules first, then optionally a different model class, then optionally a human — the gate is independent of the generator. That independence is what makes AI output trustworthy enough to commit into business records. It's "probabilistic core, deterministic gate."

Interview answer: "An LLM can't be its own judge — a confidently wrong output would pass its own check. So generation and verification are separate lanes. Deterministic business rules run first and short-circuit on failure; only then does an AI verifier (a different model class) or a human verifier run. The generator never certifies itself."

Why an in-place correction loop with attempt_seq, not versioned LO ids?

Humans on the platform recurse by creating new versioned LOs (LO4.1 → LO4.2). An AI agent that iterates five times in three seconds can't spawn five versioned LOs — it would pollute the design space and break every downstream reference. So AI attempts are recorded **in-place**: each attempt is a row in `lo_input_execution_agent` with an incrementing `attempt_seq`, the failed ones carry a

`failure_report`, and the **latest passing value is mirrored** to the canonical `lo_input_execution` that downstream readers use. You get full attempt-by-attempt audit without touching the versioning model.

Interview answer: "Human recursion creates versioned LO ids; that doesn't fit an agent looping five times a second. I kept attempts in-place with an attempt_seq and mirrored the current value to the canonical table, so downstream code reads one value while I still have every attempt audited."

Why attribute-level RBAC on top of existing LO-level RBAC?

LO-level RBAC answers "can this principal run this LO?" That's too coarse for an agent. An invoice-extraction agent should read the line-item attributes but never the approver's salary field, and it should be allowed to *write* the extracted fields but not *source* others. So permissions go down to the **attribute**, with a new `source` permission type beside read/write. It reuses the existing vertical `system_permissions` table (add an `attribute` resource_type) rather than a parallel system — extend, don't replace. The agent's input set is filtered through this before it ever reaches the prompt, so it physically cannot see out-of-scope data.

Interview answer: "Object-level access isn't enough for an agent — it needs field-level scope. I added attribute-level rows to the existing permission table with a new 'source' permission, and the agent's inputs are RBAC-filtered before prompt assembly, so it can't even see attributes outside its grant."

Why sibling tables (`_agent`) instead of altering existing tables?

The platform already runs HUMAN and DIGITAL LOs in production against canonical execution tables. Structurally changing those risks every existing workflow. So all AI state lives in **new sibling tables** suffixed `_agent`, and the only touches to canonical tables are clearly-marked **additive, nullable columns** (`permitted_sources`, NULL = legacy). This makes the whole feature additive and reversible — you can disable the AI runtime and the platform behaves exactly as before.

Interview answer: "I never altered the canonical execution tables — production HUMAN/DIGITAL flows depend on them. All agent state went into sibling tables, and the only changes to existing tables were nullable additive columns defaulting to legacy behavior. The feature is fully reversible."

Why a gRPC model gateway, stateless and horizontally scalable?

Every inference call goes through **one gRPC gateway** instead of each service calling a provider SDK. gRPC (HTTP/2, binary protobuf) is low-latency for internal service-to-service traffic on a hot path. Centralizing it means provider choice, routing by `model_class` (instruct/thinking/vision), retries, and observability live in one place. And because the gateway holds **no per-request state**, you scale it by adding replicas behind a load balancer — no sticky sessions.

Interview answer: "One gRPC gateway fronts all providers behind a single contract and routes by model class. gRPC for the low-latency binary protocol on the inference hot path, stateless so scaling is just more replicas."

Why verb-based routing via MCP, not pinning a model on the LO?

If a designer hardcodes "use model X" on an LO, every model upgrade means editing every LO. Instead the LO declares **intent** — a verb like `generate_image` — and an MCP-backed router resolves that verb to a concrete agent at runtime using a **most-specific-match** rule (exact → tenant → global → default). The LO stays portable across model upgrades, and a tenant can override routing without touching LO design. Exposing the router as an **MCP server** means design-time tools and the engine use the same protocol.

Interview answer: "Designers declare what the LO does — a verb — not which model. An MCP-backed router resolves verb → agent at runtime, most-specific match wins. Swap v3 for v4 centrally and every LO follows with no change."

Why pre-registered NFs as tools, not free-form tool creation?

An agent with unrestricted tool creation is a security and audit nightmare. Here the agent can only invoke **NFs that were pre-registered and explicitly granted** to it (`agent_tool_grants`), each with a JSON-schema parameter contract validated on every call, and each invocation audited in and out. It gets real tool use without the ability to invent uncontrolled actions.

Interview answer: "The agent's tools are a whitelist of pre-registered functions it's been granted, each with a schema-validated parameter contract and full invocation audit. It gets tool use without the power to do anything it wasn't explicitly allowed."

Why three memory layers?

Different memory has different lifetimes and purposes. **Scratch** (reasoning trace, raw output) is only useful for debugging the current run — pruned after 7 days. **Episodic** (per user × GO × LO completed attempts) is the medium-term record for analysis — 90 days. **Distilled** promotes recurring patterns from episodic into the RAG corpus, but only through **admin-gated curation** (requires N supporting episodes) so the knowledge base doesn't fill with noise. Cheap short-term memory, durable medium-term, curated long-term.

Why a hard iteration cap and no budget cap?

The correction loop must terminate — an agent that never satisfies the verifier can't loop forever, so there's a hard `iteration_max` (default 5), after which the LO transitions to a configured fallback. The choice was a **deterministic iteration cap over a token/cost budget cap** because iteration count is predictable and auditable per attempt, whereas a mid-generation budget cutoff leaves a half-produced, unverifiable output. Bounded attempts, each fully verified or fully discarded.

Why mirror the latest attempt to the canonical `lo_input_execution`?

Downstream consumers — the primary function, business rules, other LOs — read from the canonical `lo_input_execution`. They shouldn't need to know an AI agent is looping behind the scenes. So each accepted attempt is **mirrored** to the canonical table (upsert), giving downstream code one stable value to read while the full attempt history stays in the sibling table. Clean separation between "the agent's working history" and "the value the rest of the system sees."

PART 5: INTERVIEW Q&A — EVERY QUESTION AN INTERVIEWER CAN ASK

Q: Walk me through this at a high level.

Brane's no-code platform runs workflows as Local Objectives that were either HUMAN or DIGITAL — both deterministic. I contributed to a new AI_AGENT type. The hard part is that an LLM is probabilistic — it can be wrong or malformed — and you can't commit unreviewed AI output into business data. So the design is "probabilistic core, deterministic gate": the agent generates, then a *separate* verifier lane

(deterministic business rules → optional AI verifier → optional human) decides if it's acceptable. Pass commits; fail feeds the failure reason back and the agent retries in-place with an attempt counter, up to a hard cap. Attribute-level RBAC filters what data it can touch, a tool registry limits what functions it can call, and everything is audited in sibling tables so nothing existing was altered.

Q: What exactly did *you* build?

I contributed over two months on a large existing platform. My slices: the PostgreSQL schema — agent identity, the multi-agent LO junction, attribute-level RBAC rows, and the execution sibling tables (`lo_input_execution_agent` , `agent_run_scratch` , `agent_episodes`); LLM inferencing through the gRPC model gateway; agent-loop development; prompt assembly; and the performance/reliability work — RBAC caching, tool-invocation overhead reduction, and hardening the response parser against malformed LLM output. It was a team effort — I own those pieces and understand the whole design.

Q: Why is the verifier a separate call from generation?

Because a model can't judge itself — a confidently wrong answer would pass its own check. Separating generation from verification makes the gate independent of the generator. Deterministic business rules run first and short-circuit on failure — cheapest and most trustworthy. Only if those pass does an AI verifier (a different model class) or a human verifier run. That independence is what makes the output safe to commit.

Q: How does the retry loop actually work?

Each iteration: assemble the RBAC-filtered inputs plus the prompt, plus any prior failure reports; call the gateway; parse and schema-validate the output (a parse failure is itself a failed attempt); if it parses, persist the attempt and mirror it to the canonical table; run the verifier stack. On pass, commit the primary function and write an episode. On fail, record the failure report so the next iteration's prompt includes it — retry-with-feedback. Hard cap at `iteration_max` , then a configured fallback. Every attempt is a row with an `attempt_seq` , fully auditable.

Q: Why not just create a new versioned LO per retry, like human recursion?

Human recursion creates versioned LO ids — LO4.1, LO4.2. An agent looping five times in three seconds can't spawn five versioned LOs; it'd pollute the design space and break downstream references. So attempts are in-place with an `attempt_seq` , and the latest accepted value is mirrored to the canonical `lo_input_execution` . Downstream reads one stable value; I keep every attempt audited.

Q: Explain attribute-level RBAC and why LO-level wasn't enough.

LO-level RBAC answers "can this principal run this LO?" — too coarse for an agent. An invoice extractor should read line items but never see a salary field, and write extracted values but not source others. So permissions go to the attribute, with a new `source` permission beside read/write. I reused the existing vertical `system_permissions` table — added an `attribute` resource_type rather than building a parallel system. The agent's inputs are filtered through this before prompt assembly, so it physically can't see out-of-scope data.

Q: You mentioned RBAC caching — what was the problem and the fix?

Checking attribute permissions naively means a DB round-trip per attribute per attempt — and the same attributes get checked repeatedly across a loop. I added a per-instance cache keyed by (principal, attribute, permission, tenant). First check hits the DB; repeats hit the cache. It turned an N-queries-per-attempt pattern into roughly one query per distinct attribute for the whole run.

Q: How does tool invocation stay safe?

The agent can only call NFs that were pre-registered and explicitly granted to it via `agent_tool_grants`. Each call is grant-checked, its parameters validated against a JSON schema, audited into `lo_nested_function_input_execution_agent`, dispatched to the *existing* NF handler (I reused the platform's handlers rather than reimplementing), and the output audited out. It gets real tool use with zero ability to invent uncontrolled actions.

Q: What was the "tool invocation overhead reduction"?

The tool path resolves the tool, checks the grant, validates params, allocates an invocation sequence, writes input rows, dispatches, writes output rows. I reduced redundant lookups and round-trips in that path — resolving tool metadata and grants efficiently, allocating the sequence in one query — so a tool-heavy attempt doesn't spend most of its time in bookkeeping.

Q: "Response parser hardening for malformed LLM output" — what does that mean?

LLMs return broken JSON a meaningful fraction of the time — code fences, trailing text, missing fields. The parser schema-validates the output; a parse failure is treated as a failed attempt (with a `parse_failed` report) rather than crashing the run, so the loop can retry with feedback. Hardening it meant handling the common malformation modes and always degrading to a clean failed-attempt rather than an exception.

Q: Why a gRPC gateway instead of calling the provider directly?

One gateway fronts all providers behind a single contract, routes by model class (instruct/thinking/vision), and centralizes retries and observability. gRPC because it's low-latency binary

over HTTP/2 — right for an internal inference hot path. It's stateless, so it scales horizontally with replicas behind a load balancer.

Q: What's the verb-based MCP routing about?

Designers declare intent — a verb like `generate_image` — instead of pinning a model. An MCP-backed router resolves (verb, tenant, context) to a concrete agent at runtime, most-specific match wins: exact LO → tenant → global → default. So upgrading a model is a central routing change and every LO follows with zero edits. Exposing the router as an MCP server means design-time tooling and the engine share one protocol.

Q: What are the three memory layers for?

Scratch — reasoning trace and raw output for the current run, pruned after 7 days, for debugging. Episodic — per user × GO × LO completed attempts, 90 days, for analysis. Distilled — recurring patterns promoted from episodic into the RAG corpus, but admin-curated and requiring N supporting episodes so it doesn't fill with noise. Short-term cheap, medium-term durable, long-term curated.

Q: How did you avoid breaking the existing platform?

Everything additive. All agent state in sibling tables suffixed `_agent` ; the only changes to canonical tables were nullable columns defaulting to legacy behavior. The engine integration is a single branch — if an LO's primary agent is AI_AGENT, delegate to the controller; otherwise the existing HUMAN/DIGITAL path runs unchanged. You can disable the runtime and the platform behaves exactly as before.

Q: What happens when the agent never satisfies the verifier?

Hard iteration cap (default 5). On exhaustion, the run state goes to `exhausted` , an episode is written with a fallback outcome, and the LO transitions to its configured fallback state per the pathway config. Bounded, auditable, and it never blocks the workflow forever.

Q: What would you improve or what are the limits?

It's a two-month contribution to a large system, so I own specific slices, not the entire runtime. Honest limits: the human-verifier path blocks until a verdict, which needs careful timeout handling; the distilled-memory promotion is admin-gated by design, so long-term learning is deliberately slow; and the iteration cap is a fixed count, not adaptive to how close the last attempt was. Those are the places I'd look next.

PART 6: RESUME BULLETS — CURRENT VS OPTIMAL

Current bullets in resume

> Contributed to production AI Agent runtime for a no-code platform — worked across PSQL schema design, LLM inferencing via gRPC, agent loop development, prompt engineering, and performance optimization.
>
> Implemented PSQL schema components: agent identity tables, multi-agent LO junction tables, attribute-level RBAC permission rows, and agent execution sibling tables (lo_input_execution_agent, agent_run_scratch, agent_episodes).
>
> Integrated LLM inferencing layer via gRPC model gateway → multi-provider model routing (instruct, thinking, vision) with stateless horizontal scaling.
>
> Engineered prompt assembly for the agent loop: system prompt, grounded input injection, tool registry context, retry-with-feedback construction for the verifier correction loop.
>
> Resolved bugs and optimized performance → attribute RBAC caching, tool invocation overhead reduction, response parser hardening for malformed LLM outputs.

What's strong already

These are good — specific, technical, real table names. Keep them. The one thing they under-sell is the **design principle** that makes the work impressive: *probabilistic core, deterministic gate* (the separate verifier lane). That single idea is what separates this from "called an LLM in a workflow," and it's worth surfacing in the top bullet or the interview opener.

Optional sharpened top bullet

> Contributed to a production **AI agent runtime** for a no-code workflow platform — added an AI_AGENT Local-Objective type on a *"probabilistic core, deterministic gate"* design: the agent generates, a **separate verifier lane** (deterministic rules → AI → human) certifies before commit, with an in-place retry-with-feedback correction loop, attribute-level RBAC, and audited tool invocation. Stack: Python · PostgreSQL · gRPC · MCP.

One-line interview opener (memorize this)

> "The hard part of putting an AI agent in a workflow engine isn't calling the LLM — it's that the LLM is probabilistic and everything else assumes deterministic, trustworthy steps. So the whole design is *probabilistic core, deterministic gate*: the agent generates, a separate verifier lane certifies before anything commits, and it retries in-place with the failure reason fed back — all under attribute-level access control and full audit."

Skills section — confirm/add

AI & LLM row: `Agentic AI` ✅ · `MCP` ✅ (this project is your strongest MCP evidence — an MCP-backed agent router; move it from "thin" to "shipped") · consider adding `gRPC` visibility next to LLM inferencing.

Backend row: `gRPC` ✅ already listed — this is the project that proves it (model gateway, stateless scaling).

Data & Infra row: `PostgreSQL` ✅ — this project proves schema-design depth (junction tables, partial unique indexes, attribute-permission modeling, sibling-table strategy), not just usage.

Sources: resume (Brane Enterprises, AI Engineer · Associate Solution Leader) and `BRANE/AI_Agent_LO_Implementation_Guide (1).docx`*. Verify any specific claim against the implementation guide before quoting verbatim — this deep-dive summarizes and frames it for interviews.*